NESTLY · ENGINEERING DOCUMENTATION

Project Overview & BRD

Business requirements, scope, workflows and acceptance criteria

Contents

1 Executive summary
2 Problem, vision, goals and non-goals
3 Actors and personas
4 Product scope and customer journey
5 Functional requirements
6 Non-functional requirements
7 Business rules and booking lifecycle
8 Multi-resource booking and payments
9 User stories and acceptance criteria
10 Data, security and delivery phases
11 Risks, open decisions and success criteria
12 Glossary

1. Executive summary

Nestly is a marketplace where a customer books a property and the services around it (catering, decoration, cake, photography, music) in **one order with one payment**. Hosts own properties, providers own services, and the platform coordinates availability, checkout, payment and fulfilment between them.

The product is deliberately shaped around hard distributed-systems problems: exclusive inventory under concurrent demand, temporary holds, idempotent payments, multi-party compensation, asynchronous work, search and cloud scaling. Each business rule in this document exists to make one of those problems explicit and testable.

> **In one sentence:** discover → configure an event → validate availability → hold → pay once → confirm → fulfil → review, with correctness guaranteed for booking and payment.

2. Problem, vision, goals and non-goals

Problem. Event planning is fragmented. Customers verify venue availability, negotiate with each service provider and coordinate payments separately, and a failure in any one arrangement can collapse the whole event.

Vision. One place to discover, configure, book and coordinate a complete event experience.

Goals	Non-goals
Unified checkout for property plus services	A full accounting or ledger platform
Reliable, never-double-booked inventory	Owning or managing physical inventory
Host and provider coordination	A 10–20 service microservice architecture from day one
Fast reads at scale; safe concurrent writes	Real-time chat or negotiation between parties
Observable, recoverable AWS deployment	Multi-region active-active deployment

3. Actors and personas

Actor	Responsibilities	What they need from the product
Customer	Search, configure an event, book, pay, cancel, review	Fast discovery; availability they can trust; clear payment state
Host	Manage property listing, pricing and availability calendar	An accurate calendar that is never double-booked
Provider	Offer services; manage capacity and schedule; fulfil orders	Timely service orders; control over what they can accept
Admin	Moderate users and listings; run operations	Audit trail and intervention controls

4. Product scope and customer journey

Capability	Description
Property marketplace	Listings with images, amenities, capacity, pricing and availability
Service marketplace	Providers list catering, decoration, cake, photography, DJ/music and similar
Event builder	Combine one property with multiple services, or pick a reusable package
Composite booking	One order holds several items (property and services) with price snapshots
Payment	Idempotent payment operations, webhook handling, refunds
Operations	Notifications, reviews, host/provider workflows, admin moderation

Customer journey

Step	Customer action	System guarantee
1 Discover	Browse and search properties and services	Results may be slightly stale (eventual)
2 Build	Choose property, services or a package	Selections become order items with price snapshots
3 Validate	Proceed to checkout	Availability is re-checked against authoritative data
4 Hold	Checkout begins	Resources are held for a short, fixed time
5 Pay	Submit payment	Retries never create a second payment
6 Confirm	Receive result	Reservations are confirmed or fully released
7 Fulfil	Providers and host prepare	Notifications and provider actions are delivered reliably
8 Review	Rate completed bookings	Only eligible, completed bookings can be reviewed

5. Functional requirements

Priority uses MoSCoW: **M** must, **S** should. Every requirement maps to at least one acceptance test in section 9.

ID	Requirement	Group	Pri
FR-001	Registration, login and authenticated sessions	Identity	M
FR-002	Server-side RBAC for Customer, Host, Provider and Admin	Identity	M
FR-003	Host property CRUD and availability management	Inventory	M
FR-004	Provider/service CRUD; capacity and schedule management	Inventory	M
FR-005	Property and service search with filters	Discovery	M
FR-006	Event builder and package selection	Discovery	S
FR-007	Authoritative availability validation	Booking	M
FR-008	Temporary checkout holds that expire automatically	Booking	M
FR-009	Composite orders with multiple order items	Booking	M
FR-010	Payment success, failure, timeout and refund states	Payment	M
FR-011	Idempotent mutations and duplicate-webhook safety	Payment	M
FR-012	Saga-style compensation on partial multi-resource failure	Payment	M
FR-013	Asynchronous notifications and provider actions	Operations	M
FR-014	Reviews for eligible completed bookings	Operations	S
FR-015	Admin moderation and audit logging	Operations	M

6. Non-functional requirements

Targets below are design goals to be validated by load tests. They are not production claims.

Area	Requirement	Target to validate
Consistency	Strong for reservation and payment; eventual for search, cache, notifications	Zero double confirmations under contention tests
Performance	Cache read-heavy data; measure P50/P95/P99	Cached detail reads and search P95 within an agreed budget
Scalability	Stateless API nodes; horizontal scaling	Add nodes without code or session changes
Reliability	Retries, backoff, DLQ, idempotency, reconciliation	No message loss; duplicates cause no duplicate side effects
Security	HTTPS, JWT, RBAC, validation, secrets management, least privilege	No public data stores; secrets never in code or logs
Observability	Logs, metrics, request IDs, traces, alarms	Any failed booking traceable by one request ID
Maintainability	Modular boundaries; documented contracts	Docs match implementation at release

7. Business rules and booking lifecycle

Rule	Statement
BR-01	An exclusive resource (for example a property on a date range) can never have overlapping confirmed reservations.
BR-02	A hold has a fixed time limit and expires automatically, returning the resource to available.
BR-03	Checkout must revalidate availability before confirmation, regardless of what search showed.
BR-04	Order items snapshot price at checkout; later catalog changes do not alter an existing order.
BR-05	A successful payment must be reconciled with resource state; money never stays captured for a lost booking.
BR-06	Repeating the same payment request returns the original result; it never charges twice.
BR-07	Cancellation and refunds follow the defined policy (see open decision D-1).
BR-08	Only customers with a completed booking can review it.

Booking lifecycle

State	Meaning	Moves to	Trigger
AVAILABLE	Free to book	HELD	Customer starts checkout
HELD	Temporarily reserved with an expiry	PAYMENT_PENDING / AVAILABLE	Payment starts / hold expires or is released
PAYMENT_PENDING	Payment in flight	CONFIRMED / AVAILABLE	Payment success / failure or timeout
CONFIRMED	Authoritative reservation	CANCELLED	Cancellation under policy
CANCELLED	Released; refund per policy	—	Terminal

8. Multi-resource booking and payments

A composite booking spans independently owned resources: a host's property and several providers' services. They cannot share one database transaction, so the platform uses a **Saga**: reserve each resource with a local commit, process payment, and if a later step fails, run compensating actions for the steps already completed.

Situation	Required business outcome
All reservations and payment succeed	Order confirmed; all parties notified
One service cannot be reserved	Everything already held is released; customer told which item failed; nothing charged
Payment fails or times out	All holds released (or kept until expiry while status is ambiguous); no confirmed reservation without payment
Payment succeeds but confirmation fails	Automatic refund or retry until confirmed; never a silent loss of money or booking
Customer retries or double-clicks pay	Exactly one payment operation exists

Payment states: CREATED → PENDING → PROCESSING → SUCCESS or FAILED; SUCCESS → REFUNDED. Outcomes the provider reports ambiguously are reconciled against the provider's records.

9. User stories and acceptance criteria

Story	Given / When / Then	Reqs
Customer books a property	Given two customers request the same dates, when both check out, then exactly one is confirmed and the other receives a clear conflict message.	FR-007, 008, 009
Customer adds services	Given a property is selected, when a service is added, then it is validated for capacity and time before the order can proceed.	FR-004, 006
Customer retries payment	Given a payment was submitted, when the same request is sent again with the same key, then the existing operation is returned and no second charge occurs.	FR-010, 011
Hold expires	Given a hold reaches its time limit unpaid, then the resource becomes bookable by others.	FR-008
Partial failure	Given a property is held and a service fails, then the property hold is released and nothing is charged.	FR-012
Worker fails	Given a notification job fails, then it is retried, and after repeated failure lands in a dead-letter queue for inspection.	FR-013
Search is stale	Given search shows a resource that was just booked, when the customer checks out, then authoritative validation rejects it.	FR-005, 007
Admin moderates	Given a listing violates policy, when an admin acts, then the action is recorded with actor and time.	FR-015

10. Data, security and delivery phases

Core entities: users, properties, property_images, amenities, availability, providers, services, events, packages, package_services, orders, order_items, reservations, payments, payment_transactions, reviews, notifications, payouts, coupons, audit_logs, idempotency_keys.

Security: password hashing, JWT, RBAC, input validation, HTTPS, rate limiting, secrets management, least-privilege IAM, audit logs.

Phase	Scope	Outcome
1	React, Node/Express, PostgreSQL, auth	Users can sign in
2	Marketplace, availability, booking	Core booking works end to end
3	Redis cache, holds, locks; idempotency	Correct under concurrency and retries
4	Events, packages, Saga	Multi-resource orders with compensation
5	SQS workers, notifications, DLQ	Reliable background work
6	Docker, AWS, RDS, ElastiCache, S3/CloudFront	Running in the cloud
7	OpenSearch, read replicas, observability, k6	Scaled, measured, diagnosable

11. Risks, open decisions and success criteria

Risk	Impact	Mitigation
Provider-specific scheduling rules	Wrong availability logic	Model capacity and exclusive resources separately; validate per provider type
Stale search index	Customer sees unavailable items	Revalidate at checkout; show availability as indicative
Dependency outage (cache, search, payment)	Degraded experience	Isolate failure; booking path stays correct without cache or search
Premature distribution	Complexity and cost	Modular monolith first; extract only on measured need
Ambiguous payment outcome	Charge without booking	Pending state, reconciliation job, refund path

Open decisions to close before build

ID	Decision	Why it matters
D-1	Cancellation and refund policy (windows, partial refunds, who bears provider fees)	Drives BR-07 and payout logic
D-2	Hold duration for checkout (for example 10 or 15 minutes)	Trade-off between conversion and inventory lock-up
D-3	Payment order: authorize-then-capture versus pay-then-confirm	Changes compensation steps (void versus refund)
D-4	Exclusive versus capacity-based services (for example one photographer versus a caterer with capacity)	Determines the availability model per service
D-5	Payout timing to hosts and providers	Affects the payouts entity and reconciliation

Success criteria

- The full customer journey works on a live AWS deployment.
- Concurrent booking is correct: contention tests never produce two confirmations.
- Payment retries and duplicate webhooks produce exactly one logical outcome.
- A failed multi-resource booking leaves no orphaned holds or confirmed reservations.

- Workers retry, then dead-letter safely; queue depth and failures are visible.
- Logs and metrics make any failure diagnosable from a single request ID.

Capacity figures elsewhere in these documents are design assumptions, not production claims.

12. Glossary

Term	Meaning
Hold	Temporary inventory reservation with an expiry time
Reservation	Authoritative allocation record in the database
Saga	Workflow of local transactions with compensating actions for failure
Compensation	An action that undoes the business effect of an earlier completed step
Idempotency	Repeating a request produces one logical state change
Cache-aside	Read cache first, fall back to database, then populate the cache
DLQ	Dead-letter queue for messages that repeatedly fail processing
Eventual consistency	Data converges shortly after a change rather than instantly